Forwarding reference to specific type/template

Document #:	P2481R2
Date:	2023-12-16
Project:	Programming Language C++
Audience:	EWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Revision History

When [P2481R1] was discussed in Issaquah, three polls were taken. There was weak consensus to solve the problem as a whole, but there was clear preference to see “a new forwarding reference syntax”:

SF	F	N	A	SA
1	7	3	0	0

Over “additional syntax in the parameter declaration”:

SF	F	N	A	SA
3	2	4	1	2

So this paper focuses on trying to find a solution that satisfies that preference.

Since [P2481R0], added Circle’s approach and a choice implementor quote.

§ 2 Introduction

There are many situations where the goal of a function template is deduce an arbitrary type - an arbitrary range, an arbitrary value, an arbitrary predicate, and so forth. But sometimes, we need something more specific. While we still want to allow for deducing `const` vs mutable and lvalue vs rvalue, we know either what concrete type or concrete class template we need - and simply want to deduce *just* that. With the adoption of [P0847R7], the incidence of this will only go up.

It may help if I provide a few examples.

§ 2.1 `std::tuple` converting constructor

`std::tuple<T...>` is constructible from `std::tuple<U...>` *`cv ref`* when `T...` and `U...` are the same size and each `T` is constructible from `U` *`cv ref`* (plus another constraint to avoid clashing with other constructors that I'm just going to ignore for the purposes of this paper). The way this would be written today is:

```
template <typename... Ts>
struct tuple {
    template <typename... Us>
        requires sizeof...(Ts) == sizeof...(Us)
              && (is_constructible_v<Ts, Us&> && ...)
    tuple(tuple<Us...>&);

    template <typename... Us>
        requires sizeof...(Ts) == sizeof...(Us)
              && (is_constructible_v<Ts, Us const&> && ...)
    tuple(tuple<Us...> const&);

    template <typename... Us>
        requires sizeof...(Ts) == sizeof...(Us)
              && (is_constructible_v<Ts, Us&&> && ...)
    tuple(tuple<Us...>&&);

    template <typename... Us>
        requires sizeof...(Ts) == sizeof...(Us)
              && (is_constructible_v<Ts, Us const&&> && ...)
    tuple(tuple<Us...> const&&);
};
```

This is pretty tedious to say the least. But it also has a subtle problem: these constructors are all *overloads* - which means that if the one you'd think would be called is valid, it is still possible for a different one to be invoked. For instance:

```
void f(tuple<int&>);
auto g() -> tuple<int&&>;

void h() {
    f(g()); // ok!
}
```

Here, we're trying to construct a `tuple<int&>` from an *rvalue* `tuple<int&&>`. The desired behavior is that the `tuple<Us...>&&` is considered and then rejected (because `int&` is not constructible from `Us&&` - `int&&`). That part indeed happens. But the `tuple<Us...> const&` constructor still exists and that one ends up being fine (because `int&` is constructible from `Us const&` - which is `int&` here). That's surprising and undesirable.

But in order to avoid this, we'd need to only have a single constructor template. What we *want* is to have *some kind* of `tuple<Us...>` and just deduce the *`cv ref`* part. But our only choice today is either the code above (tedious, yet mostly functional, despite this problem) or to go full template:

```
template <typename... Ts>
struct tuple {
    template <typename Other>
        requires is_specialization_of<remove_cvref_t<Other>, tuple>
              && /* ???? */
    tuple(Other&& rhs);
};
```

How do we write the rest of the constraint? We don't really have a good way of doing so. Besides, for types which inherit from `std::tuple`, this is now actually wrong - that derived type will not be a specialization of tuple, but rather instead inherit from one. We have to do extra work to get that part right, as we currently do in the `std::visit` specification - see 22.6.7 [variant.visit]/1.

§ 2.2 `std::get` for `std::tuple`

We run into the same thing for non-member functions, where we want to have `std::get<I>` be invocable on every kind of tuple. Which today likewise has to be written:

```
template <size_t I, typename... Ts>
auto get(tuple<Ts...>&) -> tuple_element_t<I, tuple<Ts...>>&;

template <size_t I, typename... Ts>
auto get(tuple<Ts...> const&) -> tuple_element_t<I, tuple<Ts...>> const&;

template <size_t I, typename... Ts>
auto get(tuple<Ts...>&&) -> tuple_element_t<I, tuple<Ts...>>&&;

template <size_t I, typename... Ts>
auto get(tuple<Ts...> const&&) -> tuple_element_t<I, tuple<Ts...>> const&&;
```

This one we could try to rewrite as a single function template, but in order to do that, we need to first coerce the type down to some kind of specialization of `tuple` - which ends up requiring a lot of the same work anyway.

§ 2.3 `transform` for `std::optional`

The previous two examples want to deduce to some specialization of a class template, this example wants to deduce to a specific type. One of the motivation examples from "deducing `this`" was to remove the quadruplication necessary when writing a set of overloads that want to preserve `const`-ness and value category. The adoption of [P0798R8] gives us several such examples.

In C++20, we'd write it as:

```
template <typename T>
struct optional {
    template <typename F> constexpr auto transform(F&&) &;
    template <typename F> constexpr auto transform(F&&) const&;
    template <typename F> constexpr auto transform(F&&) &&;
    template <typename F> constexpr auto transform(F&&) const&&;
};
```

complete with quadruplicating the body. But with deducing this, we might consider writing it as:

```
template <typename T>
struct optional {
    template <typename Self, typename F> constexpr auto transform(this
        Self&&, F&&);
};
```

But this deduces too much! We don't want to deduce derived types (which in addition to unnecessary extra template instantiations, can also run into shadowing issues). We *just* want to know what the `const`-ness and value category of the `optional` are. But the only solution at the moment is to C-style cast your way back down to your own type. And that's not a particular popular solution, even amongst standard library implementors:

> FWIW, we're no longer using explicit object member functions for `std::expected`; STL couldn't stomach the necessary C-style cast without which the feature is not fit for purpose.

§ 2.4 `view_interface` members

Similar to the above, but even more specific, are the members for `view_interface` (see 26.5.3 [view.interface]). Currently, we have a bunch of pairs of member functions:

```
template <typename D>
class view_interface {
    constexpr D& derived() noexcept { return static_cast<D&>(*this); }
    constexpr D const& derived() const noexcept { return static_cast<D
        const&>(*this); }

public:
    constexpr bool empty() requires forward_range<D> {
        return ranges::begin(derived()) == ranges::end(derived());
    }
    constexpr bool empty() const requires forward_range<D const> {
        return ranges::begin(derived()) == ranges::end(derived());
    }
};
```

With deducing this, we could write this as a single function template - deducing the self parameter such that it ends up being the derived type. But that's again deducing way too much, when all we want to do is know if we're a `D&` or a `D const&`. But we can't deduce just `const`-ness.

§ 2.5 The Goal

To be more concrete, the goal here is to be able to specify a particular type or a particular class template such that template deduction will *just* deduce the `const`-ness and value category, while also (where relevant) performing a derived-to-base conversion. That is, I want to be able to implement a single function template for `optional<T>::transform` such that if I invoke it with an rvalue of type `D` that inherits publicly and unambiguously from `optional<int>`, the function template will be instantiated with a first parameter of `optional<int>&&` (not `D&&`).

Put differently, and focused more on the desired solution criteria, the goal is to be able to define this (to be clear, the syntax *`some-kind-of`* is just a placeholder):

```
template <class T> struct Base { };
template <class T> struct Derived : Base<T> { };

void f(some-kind-of(Base<int>) x) {
  std::println("Type of x is {}", name_of(type_of(^x)));
}

template <class T>
void g(some-kind-of(Base<T>) y) {
  std::println("Type of y is {}", name_of(type_of(^y)));
}

int main() {
  Base<char> bc;
  Base<int> bi;
  Derived<char> dc;
  Derived<int> di;

  f(bi);                // prints: Type of x is Base<int>&
  f(di);                // prints: Type of x is Base<int&>
  f(std::move(di));     // prints: Type of x is Base<int>&&

  g(bc);                // deduces T=char, prints: Type of y is Base<char>&
  g(std::move(dc));     // deduces T=char, prints: Type of y is Base<char>&&
  g(std::as_const(di)); // deduces T=int, prints: Type of y is Base<int>
        const&
}
```

Two important things to keep in mind here:

— `f` and `g` are both function templates

— The types of the parameters `x` and `y` are always some kind of `Base`, even if we pass in a `Derived`.

§ 3 The Ideas

There were several ideas suggested in the previous revision that fit the EWG-preferred criteria of “a new forwarding reference syntax”. Let’s quickly run through them and see how they stack up.

§ 3.1 `T auto&&`

The principle here is that in the same say that `range auto&&`is some kind of `range`, that `int auto&&` is some kind of `int`. It kind of makes sense, kind of doesn't. Depends on how you think about it.

tuple

```
template <typename... Ts>
struct tuple {
  template <typename... Us>
  tuple(tuple<Us...> auto&& rhs)
    requires sizeof...(Us) == sizeof...(Ts)
          && (constructible_from<
                Ts,
                copy_cvref_t<decltype(rhs), Us>
              > && ...);
};
```

optional

```
template <typename T>
struct optional {
    template <typename F>
    auto transform(this optional auto&& self, F&& f) {
        using U = remove_cv_t<invoke_result_t<F,
            decltype(FWD(self).value())>;

        if (self) {
          return optional<U>(invoke(FWD(f), FWD(self).value()));
        } else {
          return optional<U>();
        }
    }
};
```

view_interface

```
template <typename D>
struct view_interface {
    constexpr bool empty(this D auto& self)
        requires forward_range<decltype(self)>
    {
        return ranges::begin(self) == ranges::end(self);
    }
};
```

The advantage of this syntax is that it's concise and lets you do what you need to do.

The disadvantage of this syntax is that the only way you can get the type is by writing `decltype(param)` - and the only way to can pass through the `const`-ness and qualifiers is by grabbing them off of `decltype(param)`. That's fine if the type itself is all that is necessary (as it the case for `view_interface`) but not so much when you actually need to apply them (as is the case for `tuple`). This also means that the only place you can put the `requires` clause is after the parameters. Another disadvantage is that the derived-to-base conversion aspect of this makes it inconsistent with what `Concept auto` actually means - which is not actually doing any conversion.

§ 3.2 `T&&&`

Rather than writing `tuple<U...> auto&& rhs` we can instead introduce a new kind of reference and spell it `tuple<U...>&&& rhs`. This syntactically looks nearly the same as the `T auto&&` version, so I'm not going to copy it.

If we went this route, we would naturally have to also allow:

```
template <typename T> void f(T&&);  // regular forwarding reference
template <typename T> void g(T&&&); // also regular forwarding reference
```

The advantage here is that it's less arguably broken than the previous version, since it's more reasonable that the `tuple<U...>&&&` syntax would allow derived-to-base conversion.

But the problem with this syntax is what it would mean in the case where we wanted a forwarding reference to a specific type:

```
// this is a function taking an rvalue reference to string
void f(string&& x);

// this is a function template taking a forwarding reference to string
void g(string&&& x);
```

That visually-negligible difference makes this a complete non-starter. Yes, it's technically *distinct* in the same way that `Concept auto` is visually distinct from `Type` and a compiler would have no trouble doing the correct thing in this situation, but the `auto` there really helps readability a lot and having a third `&` be the distinguishing marker is just far too subtle in a language that has a lot of `&`s already.

§ 3.3 `forward T`

This is not a syntax that has previously appeared in the paper, but the idea is:

```
// this is a function template taking a forwarding reference to string
void f(forward string x);

// these declarations are equivalent in what they accept
template <typename T> void g(forward T x);
template <typename U> void h(U&& y);
```

Otherwise usage is similar to the `T auto&&` syntax above:

tuple

```
template <typename... Ts>
struct tuple {
  template <typename... Us>
  tuple(forward tuple<Us...> rhs)
    requires sizeof...(Us) == sizeof...(Ts)
          && (constructible_from<
                Ts,
                copy_cvref_t<decltype(rhs), Us>
              > && ...);
};
```

optional

```
template <typename T>
struct optional {
    template <typename F>
    auto transform(this forward optional self, F&& f) {
        using U = remove_cv_t<invoke_result_t<F,
            decltype(FWD(self).value())>;

        if (self) {
          return optional<U>(invoke(FWD(f), FWD(self).value()));
        } else {
          return optional<U>();
        }
    }
};
```

view_interface

```
template <typename D>
struct view_interface {
    constexpr bool empty(this forward D& self)
        requires forward_range<decltype(self)>
    {
        return ranges::begin(self) == ranges::end(self);
    }
};
```

Both `forward T x` and `T auto&& x` have the issue that there's no way to name the actual type of the parameter `x` other than `decltype(x)`. The advantage of `forward T x` is that, as a novel syntax, the fact that it behaves differently from `Concept auto` is… fine - it's a different syntax, so it is unsurprising that it would behave differently.

The disadvantage of `forward T x` is that it's a novel syntax - would require a contextually sensitive keyword `forward`:

```
struct forward { };
struct string { };

// function taking a parameter of type forward named string
void f(forward string);
```

```
// function taking a forwarding reference of type string named _
void g(forward string _);
```

But this probably isn't a huge problem - since

a. `forward` isn't a terribly common name for a type

b. the actual type would have to be a valid identifier in its own right (so no qualified names or template specializations), and

c. you're never going to write such a function template without actually wanting to use the parameter, so you're not going to forget to name it.

So while the novelty is certainly a disadvantage, the potential misuse/clash with existing syntax does not strike me as a big concern.

Note that Herb Sutter's CppFront has parameter labels, one of which is `forward`. You can declare a CppFront function template as:

CppFront	C++
`decorate: (forward s: std::string) = { s = "[" + s + "]"; }`	`auto decorate(auto&& s) -> void requires (std::is_same_v<CPP2_TYPEOF(s), std::string>) { s = "[" + CPP2_FORWARD(s) + "]"; }`

The CppFront code on the left compiles into the C++ code on the right. This seems equivalent, but there are two important differences between the CppFront feature and what's suggested here:

1. CppFront's declaration requires the argument to be exactly some kind of `std::string`, not any derived type. But this proposal needs `forward std::string s` to accept any type derived from `std::string` and also coerce it to be a `std::string` such that the parameter is always some kind of `std::string`.

2. CppFront's `forward` parameters forward on definite last use.

2. is actually a pretty useful aspect in its own right, since it allows you to annotate a `forward` parameter and not have to annotate the body - although it doesn't actually save you many characters. But (1) is the important difference - accepting derived types and coercing to base is the key aspect to this proposal.

§ 3.4 Non-forward reference syntaxes

For completeness, the previous revision had ideas for deducing `const`, a new kind of qualifier deduction, and Circle's approach of a new kind of parameter annotation.

§ 4 Proposal

There are only really two viable syntaxes I've come up with for how to solve this problem that are some kind of "new forwarding reference syntax":

`T auto&& x`	`forward T x`
`void f(std::string auto&& a);`	`void f(forward std::string a);`
`template <typename... Ts>` `void g(std::tuple<Ts...> auto&& b);`	`template <typename... Ts>` `void g(forward std::tuple<Ts...> b);`
`template <typename T>` `void h(T auto&& c);`	`template <typename T>` `void h(forward T c);`
`template <typename T>` `void i(T auto& d);`	`template <typename T>` `void i(forward T& d);`

These are all function template declarations:

— `f` takes a forwarding reference to `std::string` (`a` is always some kind of `std::string` *cv ref*). Types derived from `std::string` undergo derived-to-base conversion first.

— `g` takes a forwarding reference to some kind of `std::tuple<Ts...>`, deducing `Ts...`.

— `h` takes a forwarding to any type. Here, if I pass an lvalue of type `int`, while `decltype(c)` will be `int&`, `T` will deduce as `int`.

— `i` takes a forwarding reference to any type, but only accepts lvalues. Passing an rvalue is rejected (maybe passing a const rvalue should still succeed for consistency?)

All of these declarations insert an extra trailing template parameter, in the same way that `auto` function parameters do today. So `f("hello")` would fail (`"hello"` is not a `std::string` or something derived from it), but `f<std::string>("hello")` would work (as the implicit next template parameter that denotes the type of `a`).

Of these two, my preference is for **the latter**: `forward T param`. It's a new kind of deduction so having distinct syntax, the additionally makes clear the intended use of these parameters, is an advantage.

§ 5 References

[P0798R8] Sy Brand. 2021. Monadic operations for std::optional.
https://wiki.edg.com/pub/Wg21virtual2021-10/StrawPolls/p0798r8.html

[P0847R7] Barry Revzin, Gašper Ažman, Sy Brand, Ben Deane. 2021-07-14. Deducing this.
https://wg21.link/p0847r7

[P2481R0] Barry Revzin. 2021-10-15. Forwarding reference to specific type/template.
https://wg21.link/p2481r0

[P2481R1] Barry Revzin. 2022-07-15. Forwarding reference to specific type/template.
https://wg21.link/p2481r1